

// get the index into the tensor
// do actual computation

// compute $\text{ceil}(a/b)$

$\Rightarrow$ cuda-gelu: C/V: 3.070ms
cuda: 1.664ms

// Allocate empty tensor

// Determine grid (elements divided into blocks)

// Launch the kernel

$\Rightarrow$ C++ code to define the gelu function

cpp-gelu-src = "torch::Tensor gelu(torch::Tensor x) ;"

*) compile the C++ code and bind it to python

four/4

~~three~~ ways to write kernels:

✓ #1: write kernels using python-based Triton

✓ #3: write kernels using C++/C++

✓ #2: call pytorch kernel implementation API

: cuda-resources
= I(cuda-gelu-src)
cpp-sources
= I(cpp-gelu-src)
return:
cuda-gelu

How about Java to write kernel?

#1 ~~#1~~: write kernel using python programs
own (not fused)

#4: write kernels using Triton. } python

Triton: Automatic memory from openAI

coarsening

memory management

schedule within SMTs

manual schedule across SMTs

think about thread blocks

rather than threads

triton-gelu(x: torch.Tensor):

1 allocate output tensor

2 determine grids (elements divided into blocks)

3 launch triton-gelu kernel

4 back